

Looking Pipeline And Training Explained

Purpose: Defense preparation for the professor's technical questions

Focus: Pipeline, model training, hyperparameters, AdamW, and honest limitations

Date: May 18, 2026

1. What The Pipeline Does

Looking is not only a Gemini chatbot. Gemini is used for agent reasoning, but the project also has a Telegram interface, CrewAI agents, tools, a trained PyTorch model, data files, logging, and OpenStreetMap integration.

The real pipeline is:

```
Telegram user
-> Telegram bot state machine
-> CrewAI multi-agent pipeline
-> search tools
-> trained DistilBERT scoring tool
-> ranked response back to Telegram
```

The user first chooses a mode:

- Place mode.
- Leads mode.

Then the bot adds an explicit mode prefix:

```
[MODE: places] sushi restaurant Rabat open now
```

or:

```
[MODE: leads] I offer video editing for restaurants in Rabat
```

This prefix is important because it prevents the LLM from guessing the wrong mode.

2. The Agents

The project has three agents in `agents/crew\_setup.py`.

2.1 Query Orchestrator

Its job is to understand the user's raw request.

It extracts:

- Mode: places or leads.
- City.

- Category.
- Criteria such as open now, cheap, luxury, distance, rating.

It does not search and does not score. It only transforms messy user text into structured intent.

Defense answer:

> The orchestrator is the control layer. It understands the request and prepares the context for the other agents.

2.2 Discovery Specialist

Its job is to find candidate results.

Tools:

- ``search_places``
- ``search_leads``

For places, it tries OpenStreetMap/Nominatim first. If that fails or returns nothing, it falls back to ``data/places.csv``.

For leads, it searches ``data/leads.csv``.

Defense answer:

> The discovery agent is the retrieval layer. It finds candidates, but it does not decide the final ranking.

2.3 AI Match Scorer

Its job is to score candidates using the trained model.

Tool:

- ``score_match``

The scoring tool returns:

- Low Match.
- Medium Match.
- High Match.
- Confidence percentage.

Defense answer:

> The scoring agent is where the trained PyTorch model is used as a real functional tool, which is exactly what the brief asks for.

3. Brutal Honesty About The Data

Yes, some data is fake/synthetic.

Data	Real or Synthetic?	Purpose
`data/training_data.csv`	Synthetic	Train the matching model
`data/places.csv`	Synthetic	Fallback place data
`data/leads.csv`	Synthetic	Lead search demo data
OpenStreetMap results	Real	Live place search
OSM ratings/prices	Synthetic estimates	OSM does not provide ratings/prices

The correct thing to say:

> The training and leads data are synthetic. I used them because this is an academic prototype and I needed labeled query-candidate pairs to train and test the architecture. The OpenStreetMap place lookup is real, but the model evaluation is not a production claim.

Do not say:

> The model is production-ready.

Say instead:

> The model proves the training and integration pipeline. For production, I would collect real user queries, human approvals, and real labeled examples.

4. What The Model Learns

The deep learning problem is text-pair classification.

Input:

User query + candidate description

Output:

Low Match / Medium Match / High Match

Example high match:

Query: sushi restaurant Rabat open now
Candidate: Tokyo Nights - sushi restaurant in Rabat, open, 4.7 stars
Label: High Match

Example low match:

Query: sushi restaurant Rabat open now
Candidate: Elite Barber - luxury barbershop
Label: Low Match

So the model is not generating text. It is classifying relevance.

5. Why DistilBERT

Model used:

`distilbert-base-uncased`

Why this model:

- It is smaller and faster than full BERT.
- It already understands English language patterns.
- It works well for short text-pair classification.
- It can run locally on a laptop.
- It is a real PyTorch/transformers model that we fine-tuned.

Why not train from scratch:

> Training a transformer from scratch would require huge data and compute. Transfer learning is the correct approach for a small academic project.

Why not use only Gemini:

> The brief requires a trained deep learning model. Gemini helps agents reason, but DistilBERT is the trained model used as a tool.

6. Model Architecture

The architecture is:

```
Input text
-> DistilBERT tokenizer
-> DistilBERT encoder
-> CLS vector, size 768
-> Dropout
-> Linear 768 to 256
-> ReLU
-> Dropout
-> Linear 256 to 64
-> ReLU
-> Dropout
-> Linear 64 to 3
-> class probabilities
```

The three output classes are:

- Low Match.
- Medium Match.
- High Match.

The classifier head converts DistilBERT's text representation into one of these three labels.

7. Why Freeze Some Layers

In ``model/model.py``, we freeze:

- DistilBERT embeddings.
- First 4 of 6 transformer layers.

We train:

- Last 2 transformer layers.
- Custom classifier head.

Why:

- The dataset is small.
- Early transformer layers already know general language.
- Training all layers could overfit quickly.
- Freezing makes training faster and more stable.

Defense answer:

> I froze lower layers because they contain general language knowledge. I fine-tuned the upper layers because they are more task-specific.

Honest limitation:

> Freezing reduces overfitting risk, but it does not solve the main limitation: the dataset is still small and synthetic.

8. Parameters vs Hyperparameters

This distinction matters.

Parameters:

- Values learned by the model during training.
- Example: neural network weights.

Hyperparameters:

- Values chosen before training.
- Example: batch size, learning rate, epochs, dropout.

Professor-safe answer:

> Parameters are learned. Hyperparameters are design choices we set before training.

9. Hyperparameters Explained

Batch Size = 16

Meaning:

> The model processes 16 examples before updating its weights.

Why 16:

- Fits local memory.
- Common for transformer fine-tuning.
- More stable than very small batches.

Tradeoff:

- Bigger batch means more memory.
- Smaller batch means noisier training.

Epochs = 12

Meaning:

> The model sees the whole training dataset 12 times.

Why 12:

- Dataset is small.
- Training is fast.
- The classifier head needs enough passes to learn.

Honest answer:

> 12 epochs probably overfits synthetic data. For production I would use early stopping and a larger real validation set.

Max Length = 128

Meaning:

> Inputs are padded or truncated to 128 tokens.

Why 128:

- Queries and candidates are short.
- Faster than 256 or 512.
- Saves memory.

Risk:

- Very long descriptions could be cut.

Dropout = 0.3

Meaning:

> During training, 30 percent of some activations are randomly disabled.

Why:

- Reduces overfitting.
- Useful when data is small.

Why 0.3:

- It is a moderate value.
- Stronger than 0.1, which helps because the dataset is small.

Test Size = 20 Percent

Meaning:

> 80 percent training, 20 percent testing.

Actual numbers:

- 236 training examples.
- 59 test examples.

Honest limitation:

> The test set is very small and synthetic, so the result is not enough to claim real-world accuracy.

Stratify = Label

Meaning:

> Keep a similar label distribution in train and test.

Why:

- Avoid a test set with too many examples from only one class.
- Important because labels are not perfectly balanced.

Random State = 42

Meaning:

> Makes the split reproducible.

Why:

- The professor or another student can rerun and get the same split.

10. Why AdamW Optimizer

The optimizer updates model weights to reduce the loss.

We used:

```
torch.optim.AdamW
```

Why AdamW:

- It is standard for BERT and transformer fine-tuning.
- It adapts learning rates per parameter.
- It is usually more stable than plain SGD for transformers.
- It handles weight decay better than Adam.

AdamW vs Adam

Adam:

- Adaptive optimizer.
- Good, but weight decay is coupled with gradient updates.

AdamW:

- Adaptive optimizer.
- Decouples weight decay from the gradient update.
- Better regularization for transformer training.

AdamW vs SGD

SGD:

- Simpler.
- Can work well, but usually needs more careful tuning.
- Often slower and less stable for transformer fine-tuning.

Professor-safe answer:

> AdamW is the standard practical choice for fine-tuning transformers because it combines adaptive updates with decoupled weight decay.

11. Why Two Learning Rates

The code uses:

```
BERT layers: 2e-5  
Classifier head: 1e-3
```

Why small learning rate for BERT:

- BERT is already pretrained.
- We do not want to destroy its language knowledge.
- Transformer fine-tuning usually uses small learning rates.

Why larger learning rate for classifier:

- The classifier head is new.
- It starts randomly.
- It must learn faster.

Defense answer:

> I used a small learning rate for pretrained layers and a larger learning rate for the new classifier head because they start from different levels of knowledge.

12. Why CrossEntropyLoss

We predict one class out of three:

- Low.
- Medium.
- High.

This is multi-class classification.

`CrossEntropyLoss` is the standard loss for this task.

Why not MSE:

> MSE is for regression. This task is classification, so CrossEntropyLoss is appropriate.

13. Why Gradient Clipping

The code uses:

```
clip_grad_norm_(model.parameters(), 1.0)
```

Meaning:

> If gradients become too large, shrink them.

Why:

- Prevents unstable updates.
- Helps transformer fine-tuning.
- Acts as a training safety guard.

Defense answer:

> Gradient clipping prevents sudden large parameter updates and makes training more stable.

14. The 100 Percent Accuracy Problem

The recorded result is:

```
Accuracy: 100 percent  
Confusion matrix:  
Low: 15 correct  
Medium: 15 correct  
High: 29 correct
```

This means the model classified all 59 test examples correctly.

But be honest:

> This is not proof of production performance. The data is small, synthetic, and pattern-based. The model may have learned the template patterns, not robust real-world behavior.

Even more honest:

> The current code uses the test split during training to save the best epoch. Strictly, that test split behaves like validation. A stronger experiment would use train, validation, and final untouched test sets.

Better future setup:

- Train set for learning.
- Validation set for choosing hyperparameters.
- Test set for final evaluation.
- Real human-labeled examples.

15. What Is Real And What Is Not

Real:

- Telegram bot.
- CrewAI agents.
- PyTorch model code.
- Training process.
- Saved model weights.
- OpenStreetMap live search.
- JSON logs.
- Human-in-the-loop buttons.

Synthetic:

- Training examples.
- Leads database.
- CSV fallback places.
- Ratings and prices for OSM results.

Unfinished:

- GitHub repo not initialized in this folder.
- Slides not done.
- Demo video not done.
- No real user evaluation.
- HITL approvals are logged but not saved in a database.
- Error handling exists but is not production-grade.

Best honest statement:

> This is a strong academic prototype. It satisfies the architecture and model integration requirements, but it is not a production-grade search engine yet.

16. Short Defense Answers

Is the data fake?

Partially, yes. Training data, leads, and fallback places are synthetic. OpenStreetMap search is real.

Why did you use fake data?

Because we needed labeled query-candidate pairs quickly for an academic prototype. The goal was to demonstrate model training and integration.

Is 100 percent accuracy real?

It is real on the synthetic test split, but it does not prove real-world performance.

Is the model really used?

Yes. The scoring agent calls `score_match`, which loads `model/looking_model.pt` and scores each candidate.

Why AdamW?

Because it is the standard optimizer for transformer fine-tuning and handles weight decay better than Adam.

Why DistilBERT?

Because it is lighter than BERT, good for text classification, and realistic to fine-tune locally.

Why not just Gemini?

Because the brief requires a trained PyTorch deep learning model used as a tool. Gemini is only the agent reasoning backend.

What would you improve next?

Collect real labeled data, add train/validation/test split, add persistent approvals, add tests, push to GitHub, create slides and demo video.

17. Best Final Explanation

Use this if the professor asks for the whole technical explanation:

> Looking is a multi-agent Telegram assistant. The orchestrator understands the query, the discovery agent retrieves candidates, and the scoring agent ranks them using our trained DistilBERT model. The model was trained on synthetic query-candidate pairs to classify matches as low, medium, or high. I used DistilBERT because it is lightweight and suitable for text-pair classification. I used AdamW because it is standard for transformer fine-tuning. I used a small learning rate for pretrained BERT layers and a larger one for the new classifier head. The honest limitation is that the data is synthetic, so the 100 percent test accuracy proves the pipeline works but not that the model is production-ready.